CoreAxis Technologies

AI & Machine Learning Department Handbook

Internal reference for AI Engineers, Machine Learning Engineers, Data Scientists.
MLOps Engineers, and AI Research Engineers.

Section 1

Welcome to the AI & Machine Learning Department

1.1 A Message from the Department Head

Welcome to the AI & Machine Learning Department at CoreAxis Technologies. You are joining a team of engineers, scientists, and researchers whose work sits at the intersection of applied intelligence, enterprise software, and responsible innovation.

CoreAxis Technologies was founded on the belief that AI is not a product feature — it is the foundation upon which the next generation of enterprise software will be built. Every member of this department contributes directly to that mission, whether through building production RAG pipelines, developing LLM-powered workflows, engineering robust MLOps systems, or researching new approaches to computer vision and data analytics.

This handbook is the operational guide for our department. It documents the standards, policies, processes, and expectations that govern how we work. It is not a training curriculum, nor is it a theoretical overview of AI concepts. It is a living document that describes how AI work is done at CoreAxis — how we plan projects, build systems,

evaluate models, deploy to production, and maintain the integrity and safety of our solutions over time.

Every engineer and scientist in this department is expected to read this handbook thoroughly during their first week and return to it regularly as their work evolves. When in doubt about a process, a tool choice, or an ethical question, this document is your first reference.

1.2 What This Handbook Covers

This handbook covers the full lifecycle of AI work at CoreAxis Technologies — from the moment a project is approved through long-term maintenance in production. It addresses:

- How our department is structured and what each role is responsible for
- The standards and procedures for data collection, preparation, and annotation
- Development standards for classical ML models, LLMs, and RAG systems
- Evaluation, monitoring, deployment, and incident response procedures
- Responsible AI principles and the security requirements that govern all AI work
- Documentation obligations and cross-departmental collaboration protocols

1.3 How to Use This Handbook

This handbook is organised by the phases of the AI development lifecycle. Engineers working on a specific stage — for example, feature engineering or model deployment — should consult the relevant sections directly. The appendix at the end of this document contains reference tables, approved tool lists, and template links for day-to-day use.

Updates to this handbook are announced via the internal communications channel `#dept-ai-updates`. The version date on the cover page is the authoritative indicator of currency. Any employee who identifies a gap or inaccuracy should raise it with the department lead or submit a correction via the internal documentation review process.

Section 2

Department Overview

2.1 Mission and Mandate

The AI & Machine Learning Department is responsible for the research, development, deployment, and ongoing stewardship of all AI-powered capabilities at CoreAxis Technologies. Our mandate extends across the full product portfolio, including RAG systems, LLM applications, intelligent automation solutions, computer vision products, and enterprise data analytics platforms.

The department operates under CoreAxis's core mission: to build secure, scalable, reliable, and responsible AI solutions that help businesses automate workflows and improve decision-making. Every engineering decision, architecture choice, and product trade-off must be evaluated against this mission.

2.2 Operating Model

CoreAxis Technologies operates on a hybrid working model. Employees are expected to maintain regular in-office presence at either the Chennai (Head Office) or Bengaluru (Branch Office) location in accordance with their team's schedule, while retaining the flexibility to work remotely on designated days. Standard working hours are Monday through Friday, 9:00 AM to 5:00 PM IST.

The AI & ML Department works in close collaboration with Software Engineering, Information Technology, Information Security, and Finance. Cross-departmental dependencies are managed through formally defined integration points, which are documented in Section 24 of this handbook.

2.3 Guiding Principles

Production-First Thinking: Every model, pipeline, or system we build is evaluated against its production behaviour, not just its benchmark performance.

Research prototypes must include a clear path to production viability before they consume sustained engineering resources.

Contract-First Development: APIs, data schemas, and model interfaces are defined before implementation begins. This reduces integration debt and enables parallel workstreams across teams.

Reproducibility as a Standard: All experiments, training runs, and evaluation procedures must be reproducible. This is enforced through mandatory experiment tracking, version-pinned dependencies, and documented data lineage.

Security by Design: Security considerations are embedded at every phase of the AI lifecycle, not applied as an afterthought at deployment time. Client data protection is a non-negotiable requirement.

Responsible Deployment: AI systems that affect decision-making processes must include human review mechanisms, explainability outputs, and documented bias assessments before they go into production.

2.4 Department Scope

The AI & ML Department owns the following technical domains at CoreAxis:

Domain	Responsibility
LLM Applications	Design, development, and productionisation of large language model-powered features and services
RAG Systems	End-to-end retrieval-augmented generation pipelines including ingestion, retrieval, and generation
Classical ML & Statistical Models	Predictive models, classifiers, anomaly detection, recommendation systems
Computer Vision	Image/video classification, object detection, document intelligence

MLOps	Model registry, CI/CD for ML, infrastructure provisioning, monitoring and alerting
Data Science & Analytics	Exploratory analysis, statistical validation, business intelligence for AI products

Section 3

Team Structure and Roles

3.1 Department Organisation

The AI & Machine Learning Department is a cross-functional unit. Project teams are formed around product initiatives and typically include a mix of AI Engineers, ML Engineers, Data Scientists, and MLOps Engineers. An AI Research Engineer may be attached to projects that involve novel methods or emerging technology evaluation.

Each team operates under the oversight of a Team Lead, who is accountable to the Department Head. The Department Head reports directly to the Chief Technology Officer (CTO).

3.2 Role Descriptions

AI Engineer

AI Engineers at CoreAxis are responsible for designing, building, and integrating AI-powered features into product systems. This role is primarily concerned with applied AI development — taking models, APIs, and research outputs and turning them into reliable, scalable software.

Primary Responsibilities:

- Design and implement LLM-powered application layers using frameworks such as LangChain and LlamaIndex
- Build and maintain RAG pipelines — from document ingestion through chunking, embedding, retrieval, and generation

- Develop and expose AI capabilities as FastAPI services, ensuring compatibility with the Software Engineering team's integration requirements
- Write integration tests, evaluate end-to-end system behaviour, and document AI service contracts
- Collaborate with MLOps Engineers on deployment configuration, environment management, and monitoring hooks

Key Skills: Python, FastAPI, LangChain, LlamaIndex, Docker, REST API design, prompt engineering, vector database integration, logging and observability

Machine Learning Engineer

Machine Learning Engineers are responsible for the full ML development lifecycle — from data preparation through model development, evaluation, and handoff to MLOps for deployment.

Primary Responsibilities:

- Implement data preprocessing, feature engineering, and training pipelines using Python and relevant ML frameworks
- Train, tune, and evaluate machine learning models including classical ML, deep learning, and transformer-based architectures
- Manage experiment tracking using MLflow, ensuring all runs are logged with parameters, metrics, and artefacts
- Conduct model evaluation using appropriate metrics and document results in standard evaluation reports
- Package models and produce deployment-ready artefacts in coordination with MLOps Engineers

Key Skills: Python, scikit-learn, PyTorch or TensorFlow, MLflow, data pipelines, statistical evaluation, Hugging Face ecosystem, Git

Data Scientist

Data Scientists are responsible for extracting insights from data, validating AI product hypotheses, and providing analytical support across AI development projects.

Primary Responsibilities:

- Conduct exploratory data analysis (EDA) and produce findings that inform model design and feature engineering decisions
- Design and execute statistical experiments, A/B tests, and model performance analyses
- Build and maintain analytical dashboards and reports for internal stakeholders and AI product monitoring
- Validate data quality, annotate datasets, and document data lineage in accordance with department standards
- Provide business context for AI metrics and help translate model performance into product outcomes

Key Skills: Python, SQL, PostgreSQL, statistical analysis, data visualisation, Jupyter notebooks, MLflow, data storytelling

MLOps Engineer

MLOps Engineers are responsible for the infrastructure, automation, and operational systems that enable AI models to be deployed, monitored, and maintained in production at scale.

Primary Responsibilities:

- Design and manage ML infrastructure including containerised training environments (Docker, Kubernetes) and model serving platforms
- Build and maintain CI/CD pipelines for model training, validation, and deployment using GitHub Actions

- Configure model registries in MLflow and enforce model versioning and promotion policies
- Implement monitoring, alerting, and drift detection systems for production AI services
- Manage cloud infrastructure on AWS and Azure for AI workloads, including cost governance

Key Skills: Docker, Kubernetes, AWS, Azure, MLflow, GitHub Actions, infrastructure-as-code, Prometheus/Grafana, CI/CD pipelines

AI Research Engineer

AI Research Engineers evaluate emerging AI methods, prototype novel approaches, and translate research findings into production-applicable techniques. This role bridges academic research and engineering practice.

Primary Responsibilities:

- Monitor and evaluate relevant research literature, producing structured summaries for the department
- Prototype and benchmark new methods — including new model architectures, embedding strategies, and retrieval techniques — against current production approaches
- Collaborate with ML Engineers and AI Engineers to identify where research improvements can be incorporated into production systems
- Produce technical reports documenting research findings, experimental results, and recommendations
- Represent CoreAxis at relevant industry forums, conferences, and research communities where appropriate

Key Skills: Python, PyTorch, research paper comprehension, experimental design, Hugging Face, benchmarking, technical writing

3.3 Responsibilities Matrix

Activity	AI Eng	ML Eng	Data Sci	MLOps	Research
RAG pipeline development	Lead	Support	—	Support	Consult
Model training & tuning	Support	Lead	Support	—	Consult
Data analysis & EDA	—	Support	Lead	—	—
Production deployment	Support	Support	—	Lead	—
Experiment tracking	Contribute	Lead	Contribute	Support	Contribute
Research evaluation	Consult	Support	—	—	Lead
Monitoring & alerting	—	—	Support	Lead	—

Section 4

AI Development Lifecycle

4.1 Overview

All AI projects at CoreAxis Technologies follow a structured development lifecycle. This lifecycle ensures consistency across projects, enforces quality gates at critical decision points, and provides auditability for AI systems throughout their operational life.

The lifecycle is not a rigid waterfall process. Teams operate iteratively within each phase, and some phases may run concurrently for certain project types. However, the gates between phases are non-negotiable — progression to the next phase requires documented approval.

4.2 Lifecycle Phases

Phase 1: Problem Definition and Feasibility

Every AI project begins with a structured problem definition. The team, in collaboration with the requesting stakeholder, produces a Project Brief that captures:

- The business problem being addressed and the decision it will improve or automate
- Success criteria — what measurable outcomes will define the project as successful
- Data availability assessment — what data exists, who owns it, and whether it is sufficient for the intended task
- Ethical and regulatory considerations specific to the domain and data
- A feasibility verdict, which may be: *Proceed*, *Proceed with Conditions*, or *Not Feasible*

Gate: The Project Brief must be reviewed and approved by the Department Head before proceeding.

Phase 2: Data Acquisition and Preparation

Upon project approval, the team initiates the data acquisition process in accordance with the standards described in Sections 5 and 6. This includes sourcing data, validating its quality, applying cleaning procedures, and producing a documented Data Card for the dataset.

For supervised learning tasks, annotation is conducted under the guidelines in Section 7 before this phase is considered complete.

Gate: A completed Data Card and annotation validation report must be on file before model development begins.

Phase 3: Feature Engineering and Model Development

Feature engineering and model development proceed in parallel where possible. This phase encompasses exploratory analysis, feature selection, baseline model construction, and iterative model development. All experiments are tracked in MLflow from the first training run.

Gate: A Baseline Evaluation Report is required before the team commits to a model architecture for the production candidate.

Phase 4: Evaluation and Validation

The production model candidate is evaluated comprehensively in accordance with Section 15. This includes performance evaluation on held-out test sets, fairness assessments, edge case analysis, and a responsible AI review.

Gate: A signed Model Evaluation Report is required before the model is registered in the model registry and promoted to staging.

Phase 5: Deployment

Deployment is executed under the procedures in Section 20. This includes containerisation, environment configuration, staging deployment and validation, production release, and rollback plan documentation.

Gate: Staging validation sign-off is required before production deployment is initiated.

Phase 6: Monitoring and Maintenance

Following deployment, the model enters the maintenance phase described in Section 21. This includes ongoing performance monitoring, drift detection, retraining triggers, and periodic review cycles.

Models in production are formally reviewed every six months or when a significant drift event is detected, whichever comes first.

4.3 Project Documentation Obligations

Each project must maintain a living project folder in the shared documentation system. This folder must contain, at minimum, the Project Brief, Data Card, Experiment Log (exported from MLflow), Model Evaluation Report, Deployment Runbook, and a Monitoring Dashboard link. See Section 23 for full documentation requirements.

4.4 Lifecycle for RAG and LLM Projects

RAG and LLM-based projects follow the same lifecycle structure but include additional gates specific to generative AI:

Hallucination Risk Assessment is required before staging deployment

Prompt Evaluation Report (covering adversarial inputs, jailbreak attempts, and output quality) must be completed before production release

Human Review Threshold Configuration — the confidence or relevance threshold below which outputs are routed to a human reviewer — must be documented and approved

Section 5

Dataset Collection Standards

5.1 Data Collection Principles

The quality, provenance, and legal standing of training and evaluation datasets directly determines the quality and trustworthiness of AI models built from them. CoreAxis Technologies enforces strict standards on all data collection activities. No data may be used in an AI project without a documented and approved collection process.

The following principles govern all data collection at CoreAxis:

Purpose Limitation: Data must be collected for a specific, documented purpose. Re-use of a dataset for a different project requires a new data usage assessment.

Consent and Licensing: All data used in training or evaluation must have a documented legal basis. For client data, this basis must be explicit contractual permission. For third-party datasets, a valid open-source or commercial licence must be recorded in the Data Card.

Minimisation: Collect only the data necessary for the task. Avoid collecting personal or sensitive attributes that are not required for model functionality.

Accuracy: Collected data must be appropriate and representative for the intended use case. Teams are responsible for identifying and documenting known biases, gaps, and limitations in their datasets.

5.2 Approved Data Sources

Data may be sourced from the following categories, subject to the requirements listed:

Source Type	Requirements
Internal proprietary data	Approved by data owner; documented in project plan
Client-provided data	Written client consent; DPA signed; stored in isolated, encrypted environment
Licensed third-party datasets	Licence reviewed and approved; licence file stored in project repository
Open-source datasets (Hugging Face, etc.)	Licence verified (confirm allows commercial use if applicable); cite original source
Web-scraped data	Prior approval from Department Head required; robots.txt compliance mandatory; legal review required
Synthetic data	Generation method documented; quality validation performed

5.3 Data Card Requirements

Every dataset used in an AI project must have a Data Card on file before the dataset is used in any training, evaluation, or development activity. The Data Card is a structured document that captures the following:

Dataset Name and Version: A unique identifier and version tag consistent with the project repository

Collection Date and Method: When and how the data was collected

Source and Licence: Origin of the data and the legal basis for its use

Description: Summary of dataset contents, domain, and format

Size: Record count, file size, and any notable structural properties

Features / Columns: List of all attributes with types and descriptions

Known Limitations and Biases: Documented gaps, under-represented populations, or known quality issues

PII Status: Whether the dataset contains personally identifiable information and what masking or anonymisation has been applied

Approved Projects: List of projects that are explicitly authorised to use this dataset

Data Owner: Name and role of the individual accountable for this dataset

Data Card templates are available in the internal documentation repository under `/templates/ai/data-card.md`.

5.4 Client Dataset Handling

Client data is subject to heightened protections at CoreAxis. The following requirements apply without exception:

Client datasets must be stored in project-specific, encrypted storage partitions in the designated cloud environment (AWS S3 with SSE-KMS or Azure Blob Storage with Customer-Managed Keys)

Access to client data is granted on a least-privilege basis via role-based access control. Access logs must be retained for a minimum of 12 months

Client data must not be transferred to any system, environment, or third-party service not explicitly covered by the relevant Data Processing Agreement (DPA)

Client data used in model training must be deleted from training environments within 30 days of project completion unless a longer retention is explicitly required and documented

Any incident involving suspected exposure or breach of client data must be reported immediately to the Information Security team and the Department Head

Warning: Feeding client data into external AI APIs (including OpenAI, Anthropic, Google, or others) without explicit written approval from both the client and the Information Security team is a serious policy violation and may constitute a data protection breach.

Section 6

Data Preparation and Cleaning

6.1 Standards and Expectations

Data preparation is not a preliminary step — it is a core engineering activity that directly determines model quality and system reliability. At CoreAxis, data preparation pipelines are treated with the same rigour as application code: they are version-controlled, reviewed, and documented.

All data preparation scripts must be written in Python, committed to the project Git repository, and executed in reproducible environments. Jupyter notebooks may be used for exploratory preparation but must be refactored into pipeline scripts before the preparation work is considered complete for production use.

6.2 Standard Preparation Procedures

Schema Validation

Before any transformation is applied, raw data must be validated against a documented schema. Schema validation checks include: expected column names and types, acceptable value ranges, cardinality constraints, and required field completeness. Validation failures must be logged with record-level detail, not silently dropped.

Duplicate Handling

Exact duplicates are removed. Near-duplicate detection is applied where semantically redundant records could bias training. The deduplication strategy and threshold must be documented in the Data Preparation Report.

Missing Value Treatment

Missing values are handled according to a documented strategy for each feature. Acceptable strategies include: imputation (mean, median, mode, or model-based), removal of affected records (with documentation of the volume impact), or flagging as a separate category where missingness is itself informative. Random imputation without documented justification is not acceptable.

Outlier Analysis

Outliers in numerical features are identified using statistical methods (IQR, z-score, or domain-specific bounds) and treated according to a documented decision: cap/clip, remove, or retain with flagging. All outlier treatment decisions must be documented.

Format Normalisation

Text data is normalised consistently: encoding is standardised to UTF-8; trailing whitespace and control characters are removed; date/time fields are parsed to a canonical format; categorical values are standardised (case, spacing, known synonyms).

PII Scrubbing

Where datasets contain personal information that is not required for the task, PII scrubbing is applied before the data is used in any training or development pipeline. Scrubbing methods must be validated: test the scrubber against known PII patterns and document the false-negative rate.

6.3 Data Preparation Report

Upon completion of data preparation, the responsible engineer produces a Data Preparation Report. This report documents the transformations applied, volume changes (rows in vs. rows out), decisions made at each step, and a summary of the resulting dataset's statistical properties. This report is stored alongside the Data Card in the project documentation folder.

6.4 Data Versioning

Prepared datasets must be versioned using a consistent naming convention: `<project_id>_<dataset_name>_v<major>.<minor>_<YYYYMMDD>`. Each version is immutable — prepared data files are not modified in place. A new version is created whenever preparation logic changes or the source data is updated.

Dataset versions and their associated preparation scripts are tracked in the project's MLflow experiment or in the project's Git repository (for smaller datasets suitable for Git LFS). Large datasets (>1 GB) are stored in the designated cloud storage path and catalogued in the project Data Registry.

6.5 Data Pipeline Code Standards

Preparation pipeline code must meet the following standards:

- Functions must be unit-tested for known edge cases (empty inputs, nulls, boundary values)
- Pipeline runs must be logged: input record count, transformation steps applied, output record count, and any validation failures

Side effects (writing to storage, sending notifications) must be isolated and configurable — pipelines must support a dry-run mode

Dependencies must be pinned in a `requirements.txt` or `pyproject.toml` that is committed to the repository

Section 7

Data Annotation Guidelines

7.1 Annotation as a Core Process

For supervised learning and evaluation dataset construction, annotation quality is the principal determinant of model quality. CoreAxis treats annotation as an engineering discipline, not a clerical task. Annotation processes must be designed, documented, and quality-controlled with the same rigour applied to software development.

7.2 Annotation Planning

Before any annotation work begins, the team must produce an Annotation Plan that specifies:

Task Definition: A precise description of what annotators must decide and what criteria define each label

Label Schema: An exhaustive list of all labels, categories, or entities to be annotated, with definitions and examples for each

Annotation Tool: The tool to be used (internal tooling, Label Studio, or an approved third-party service)

Quality Targets: The minimum inter-annotator agreement (IAA) score required (see 7.4)

Annotator Selection: Who will perform the annotation and what background knowledge they require

Pilot Phase: A defined pilot batch that will be completed and reviewed before full-scale annotation begins

7.3 Annotation Guidelines Document

Every annotation project must have a written Annotation Guidelines Document. This document is the definitive reference for annotators and must include:

- Clear, unambiguous definitions for each label class
- Positive and negative examples for every label — including edge cases and boundary cases
- An explicit decision tree for ambiguous cases
- Instructions on what to do when a sample does not fit any label (flag for review, choose the closest label, etc.)
- Examples of common annotator errors to avoid

The Annotation Guidelines Document must be reviewed and approved by the project lead before annotation begins. Any changes to the guidelines during annotation (which should be avoided where possible) must be versioned and communicated to all annotators before they continue.

7.4 Inter-Annotator Agreement

All annotation projects that involve more than one annotator must measure inter-annotator agreement (IAA). The required minimum IAA scores by task type are:

Task Type	Metric	Minimum Acceptable
Binary classification	Cohen's Kappa (κ)	$\kappa \geq 0.70$
Multi-class classification	Cohen's Kappa (κ)	$\kappa \geq 0.65$
Named entity recognition	Span-level F1	≥ 0.80
Sequence labelling	Token-level Agreement	≥ 0.80
Relevance rating (1–5 scale)	Weighted Kappa	$\kappa \geq 0.60$

LLM output quality rating	Percentage Agreement	≥ 75%

If IAA falls below the required threshold, the annotation guidelines must be revised and a new pilot batch conducted before full annotation resumes. Datasets annotated below the minimum IAA threshold must not be used in model training without written approval from the Department Head and a documented justification.

7.5 Quality Control Procedures

Gold Standard Samples: A set of pre-annotated gold standard samples is embedded throughout the annotation batch. Annotators whose accuracy on gold samples falls below 85% are flagged for remediation.

Random Audit: A random 5–10% sample of completed annotations is reviewed by a senior team member for quality.

Adjudication: Disagreements between annotators are resolved through an adjudication process: the item is reviewed by a third annotator or a domain expert, and the final label is documented with the resolution rationale.

7.6 Annotation for LLM Evaluation

When annotating outputs from LLMs for evaluation purposes (e.g. rating factual accuracy, helpfulness, or harmlessness), the following additional requirements apply:

Annotators must not be aware of which model or prompt produced which output (blind evaluation)

Each output must be rated by a minimum of two independent annotators

The evaluation rubric must define each dimension (accuracy, relevance, safety, fluency) with explicit rating anchors

Annotator fatigue is a known source of quality degradation — no annotator should rate more than 150 LLM outputs per session without a break

Section 8

Feature Engineering Best Practices

8.1 Principles of Feature Engineering at CoreAxis

Feature engineering bridges raw data and model training. At CoreAxis, features are treated as software artefacts: they are versioned, documented, tested, and reused where possible. Ad hoc feature construction that is not reproducible and not tracked is not acceptable in any model that is intended for production.

8.2 Feature Documentation

Every feature used in a production model must be documented in a Feature Specification. This document records the feature name, description, data type, source field(s), transformation logic (as code or a clear pseudocode description), expected value range, known edge cases, and the model(s) it is used in.

Feature Specifications are stored in the project's documentation folder and referenced in the Model Card at deployment time.

8.3 Feature Engineering Practices

Encoding Categorical Variables

The choice of encoding strategy must be documented and justified. Accepted strategies include one-hot encoding (for low-cardinality nominals), label encoding (for ordinal features with meaningful order), target encoding (with appropriate cross-validation safeguards to prevent leakage), and embedding-based encoding (for high-cardinality text or ID features). Do not default to label encoding for non-ordinal categories — this introduces unintended ordinal assumptions into the model.

Numerical Transformations

Scaling and normalisation are applied consistently across training and inference. All scaling parameters (mean, standard deviation, min/max) are fitted on the training set only and applied to validation and test sets as transforms. These parameters are

serialised and stored with the model artefact. Log transforms, power transforms, and other normalisations are documented with their rationale.

Text Features

For classical ML models using text, features are generated using TF-IDF, n-gram models, or embedding-based approaches. For deep learning and LLM-based models, raw text is passed to the model's tokeniser. When using pre-trained embeddings, the embedding model version is pinned and documented. Text preprocessing decisions (lowercasing, tokenisation, stopword removal) are consistent between training and inference.

Temporal Features

When working with time-series or time-aware data, strict temporal splitting must be applied: training data precedes validation data which precedes test data in time. Feature engineering on temporal data must not use future information — all rolling windows, lagged features, and aggregations are computed using only information available at the prediction time.

Preventing Data Leakage

Data leakage — the accidental introduction of target-correlated information that would not be available at inference time — is one of the most common sources of inflated evaluation results in ML projects. All feature engineering decisions must be reviewed for potential leakage, and a leakage checklist must be completed before the model evaluation phase begins. Common leakage sources include: fitting scalers on the full dataset before splitting, using post-event features as predictors, and including ID fields that correlate with the target in small datasets.

8.4 Feature Reuse and the Feature Registry

Features that have been engineered for one project may be suitable for reuse in future projects. When a feature is validated in production and deemed generally applicable, the

implementing engineer registers it in the department Feature Registry with its full specification. Engineers starting new projects should consult the Feature Registry before implementing common transformations from scratch.

8.5 Feature Testing

Feature engineering code must include automated tests. Required test coverage includes:

- Correct output type and shape for valid inputs
- Correct handling of null and missing values
- Correct handling of out-of-range or unexpected values
- Consistency: the same input always produces the same output (for deterministic features)
- No data leakage in temporal features (verifiable via test with shuffled dates)

Section 9

Model Development Standards

9.1 General Development Standards

Model development at CoreAxis follows engineering best practices adapted for the ML context. Code is version-controlled in Git on GitHub. Experiments are tracked in MLflow. Models are developed against documented requirements and evaluated against agreed metrics.

Every model development project begins with a documented baseline: the simplest possible model that addresses the task (e.g. a logistic regression classifier, a mean-prediction regressor, or a rule-based system). This baseline establishes the performance floor that a more complex model must meaningfully exceed to justify the added operational overhead.

9.2 Model Selection Criteria

Model architecture selection must be justified against the following criteria:

Task Fit: Is the model type appropriate for the task structure (classification, regression, clustering, generation, etc.)?

Data Volume: Does the available data volume support the chosen architecture's sample complexity?

Inference Constraints: What are the latency, throughput, and memory requirements at inference time? Does the model meet them?

Explainability Requirement: Does the use case require interpretable predictions? If so, black-box models require additional explainability layers.

Maintenance Cost: A more complex model is acceptable only if its performance advantage justifies the higher operational and retraining cost.

9.3 Training Environment

All model training runs must be executed in containerised environments (Docker) to ensure reproducibility. Training containers are configured with pinned dependency versions. Training runs on cloud infrastructure (AWS SageMaker or Azure Machine Learning) for large-scale jobs; local or small-scale training may use department-provisioned GPU workstations.

All training jobs are parameterised — hyperparameters are not hardcoded. Parameters are logged to MLflow at the start of each training run.

9.4 Hyperparameter Tuning

Hyperparameter tuning is conducted using systematic search strategies. Grid search is acceptable for small parameter spaces. Random search or Bayesian optimisation is used for larger search spaces. All tuning runs are tracked in MLflow with the parameter ranges searched and the best configuration found.

Hyperparameter optimisation must be conducted using the validation set only. The test set must not be used for any tuning decision — it is reserved exclusively for final model evaluation.

9.5 Reproducibility Requirements

Every training run must be reproducible. This requires:

- Random seeds set and logged for all sources of randomness (Python, NumPy, framework-level)
- Dataset version pinned in the MLflow run
- All dependency versions pinned
- Training code committed to Git; the commit hash is logged in MLflow
- The full training configuration (parameters, data path, preprocessing settings) is stored as an MLflow artefact

9.6 Model Artefact Standards

Trained models are saved in a format compatible with MLflow's model logging API. Alongside the model weights, the following artefacts must be logged:

- Feature preprocessor(s) with fitted parameters (serialised as a pipeline)
- A schema file describing expected input features and types
- A sample input/output pair for sanity checking
- Evaluation metrics on the test set
- The Model Card (see Section 23)

9.7 Code Review for Model Code

All model training code and preprocessing pipeline code must be reviewed by at least one other engineer before it is used to produce the model artefact submitted for production. Code review for ML code should focus specifically on: data leakage, correct

train/validation/test splitting, correct metric calculation, and correct serialisation of preprocessing parameters.

Section 10

Large Language Model (LLM) Development Standards

10.1 Scope

This section covers the development standards for AI applications built on top of large language models — whether proprietary models accessed via API, open-source models hosted on-premise, or models fine-tuned internally. It applies to all AI Engineers and ML Engineers working on LLM-powered features, workflows, or products.

10.2 Model Selection and Approval

LLM selection is a formal decision that must be documented and approved. The following categories of models are available to project teams:

Category	Examples	Approval Required
Approved self-hosted open-source	Llama 3, Mistral, Phi, Qwen	Team Lead
Approved third-party API	OpenAI GPT-4o, Anthropic Claude, Gemini	Team Lead + Security Review
Internally fine-tuned models	Any of the above, fine-tuned on internal data	Department Head
Novel/experimental models	New open-source releases, beta APIs	Department Head + Research Engineer review

Self-hosted models are run using Ollama (for local/development use) or deployed as containerised inference services on Kubernetes for production. Third-party API usage must comply with the data handling policies in Section 5.

10.3 LLM Application Architecture

LLM applications at CoreAxis use LangChain or LlamaIndex as the primary orchestration frameworks. Direct API calls to LLM endpoints without an orchestration layer are acceptable only for simple, single-turn, non-stateful use cases.

All LLM applications must implement the following components:

Input Validation Layer: Validates and sanitises user or system inputs before they are passed to the LLM

Prompt Management: Prompts are maintained as versioned template files, not as hardcoded strings in application code

Output Parsing and Validation: LLM outputs are parsed into structured types wherever possible. Validation is applied to check that outputs conform to expected formats and safety constraints

Logging: Inputs, prompts, outputs, and latency are logged for every LLM call in the production environment (with appropriate PII filtering)

Fallback Handling: A defined fallback strategy is implemented for cases where the LLM call fails (timeout, error, rate limit) or the output fails validation

10.4 Fine-Tuning Standards

Fine-tuning a base LLM is a resource-intensive activity that requires formal justification. Fine-tuning is approved only when prompt engineering and RAG are insufficient to meet the task requirements.

Approved fine-tuning approaches include: full fine-tuning (for small models), parameter-efficient fine-tuning (PEFT) using LoRA or QLoRA, and instruction tuning on curated datasets. All fine-tuning runs must be tracked in MLflow with the same rigour applied to classical ML training runs. Fine-tuned model weights are stored in the organisation's model registry, not in individual engineers' personal storage.

10.5 Context Window Management

Context window usage is a key cost and performance variable in LLM applications. Engineers must implement explicit context management strategies, including: selective document retrieval to reduce context length, conversation history truncation for multi-turn applications, and token budget tracking in application logs. Context window overflows must be handled explicitly — silent truncation that changes the semantics of the input is not acceptable.

10.6 Caching

LLM inference is expensive. Caching strategies must be evaluated for every LLM-powered feature. Semantic caching using Redis is the preferred approach for applications where similar queries are likely. Exact-match caching is appropriate for deterministic, frequently repeated prompts. Cache TTL settings must be documented and appropriate to the rate of knowledge change in the relevant domain.

10.7 Cost Management

Token consumption and API costs are tracked per project. Engineers must configure cost alerts for all third-party API integrations. Projects that exceed their monthly API cost budget require team lead approval before the overage is authorised. Cost estimates must be included in project briefs for any LLM-powered feature.

Section 11

Retrieval-Augmented Generation (RAG) Development Standards

11.1 Overview

Retrieval-Augmented Generation (RAG) is a core architecture pattern at CoreAxis Technologies, used extensively in enterprise knowledge bases, document intelligence products, and intelligent automation workflows. This section defines the internal

standards for all RAG system development, from document ingestion through evaluation.

11.2 Document Ingestion Standards

Document ingestion is the first stage of any RAG pipeline. The following standards apply to all ingestion implementations:

Format Support: Ingestion pipelines must explicitly declare supported file formats (PDF, DOCX, HTML, plain text, Markdown, etc.) and handle unsupported formats with a descriptive error rather than silent failure

Extraction Validation: Extracted text must be validated for completeness. Scanned PDFs, image-heavy documents, and encrypted files require separate handling paths with documented limitations

Metadata Extraction: Source file name, document type, date (document creation date, ingestion date), author (where available), section/page reference, and any domain-specific identifiers must be extracted and stored alongside the text content

Idempotency: Ingestion pipelines must be idempotent — re-ingesting a document that already exists must update, not duplicate, the document in the vector store

Audit Log: Every ingestion event (new document, update, delete) is recorded in the ingestion audit log with a timestamp, document identifier, and the user or process that triggered the event

11.3 Chunking Strategies

Chunking — the division of documents into smaller segments for embedding and retrieval — has significant impact on retrieval quality. The chunking strategy must be documented and validated for each RAG system.

Strategy	Appropriate Use
Fixed-size token chunks (512–1024 tokens)	General-purpose, when document structure is unknown or inconsistent
Sentence-level chunking	When queries are likely to match at the sentence level; requires reliable sentence boundary detection
Paragraph / semantic chunking	Structured documents where paragraph breaks align with semantic units; preferred when feasible
Document-level chunking (small docs)	For documents small enough to fit in the LLM context (emails, short reports)
Hierarchical / parent-child chunking	When both coarse-grained context (for re-ranking) and fine-grained retrieval are needed

Overlap: A context overlap of 10–20% is applied between consecutive chunks to reduce the risk of splitting a relevant passage across chunk boundaries. The overlap size is documented and consistent across a given corpus.

Validation: Chunking quality is validated by human review of a random sample of chunks — confirming that chunks are semantically coherent and that retrieval for known queries returns the expected chunks.

11.4 Metadata Management

Metadata is stored alongside chunk embeddings in the vector database. Metadata schemas are defined before the ingestion pipeline is built and treated as a versioned contract. Changes to the metadata schema require a re-ingestion plan.

Required metadata fields for all RAG systems:

- `source_id` — unique identifier for the source document
- `source_name` — human-readable document name
- `chunk_index` — position of the chunk within the source document

`ingested_at` — ISO 8601 timestamp of ingestion

`content_type` — document type (e.g. policy, report, manual)

Domain-specific metadata (e.g. client ID, product version, regulatory category) is added per project and documented in the system's RAG Architecture Document.

11.5 Embedding Generation

Embedding model selection follows the guidelines in Section 13. Once selected, the embedding model is pinned to a specific version for the lifetime of the index. Changing the embedding model requires a full re-ingestion and re-embedding of the corpus.

Embedding generation is performed in batches to manage API rate limits and resource consumption. Batch size is configured per model and documented. Failed batches are retried with exponential backoff; persistent failures are logged and flagged for investigation.

11.6 Vector Database Standards

See Section 14 for vector database selection and management. Within RAG systems, the following operational standards apply:

Collections/indexes are named using the convention:
`<project_id>_<corpus_name>_<embedding_model_short>_v<N>`
Index configurations (distance metric, index type, HNSW parameters where applicable) are documented and consistent within a project
Index backups are configured and tested before the system goes into production

11.7 Retrieval Strategies

The retrieval strategy is selected based on the query types and quality requirements of the specific RAG application. The following strategies are approved for use at CoreAxis:

Dense Retrieval (Semantic Search)

Standard cosine similarity search against the embedding index. Appropriate as the baseline for most RAG applications. Top-k is configured per application (typically k=5–20 before re-ranking).

Sparse Retrieval (BM25 / Keyword Search)

BM25-based keyword search is integrated using the PostgreSQL full-text search capability or a dedicated search index. Particularly effective for queries containing specific technical terms, product codes, or named entities that may not be well-represented in embedding space.

Hybrid Search

Hybrid search combines dense and sparse retrieval results, merging them using Reciprocal Rank Fusion (RRF) or a weighted combination. Hybrid search is the preferred approach for production RAG systems where query types are diverse. The weighting between dense and sparse components is treated as a tunable parameter and validated on a representative query set.

Multi-Query Retrieval

For complex queries that may be answered by retrieving from multiple sub-queries, a multi-query retrieval strategy generates reformulated query variations, retrieves for each, and deduplicates results before passing to the generation step. This strategy is used when single-query retrieval consistently misses relevant passages on complex tasks.

11.8 Re-Ranking

Retrieved candidates are re-ranked using a cross-encoder model before being passed to the generation step. Re-ranking significantly improves retrieval precision at the cost of additional latency. The re-ranker model is selected based on task domain and latency requirements, and its version is pinned and documented.

Re-ranking is mandatory for any RAG system used in a compliance-sensitive context or where retrieval quality is a patient safety, financial, or legal concern.

11.9 Source Attribution

All RAG system responses must include source attribution. The generation prompt must instruct the LLM to cite the sources used in its response. The system must pass source metadata (at minimum: document name and section/page reference) to the prompt, and the final response must include a structured citation block. Source attribution is validated as part of the RAG evaluation process (Section 11.11).

11.10 Hallucination Reduction

RAG architecture reduces but does not eliminate LLM hallucination. CoreAxis applies the following additional measures:

Grounding prompts: The system prompt explicitly instructs the model to answer only from the provided context and to indicate when the context does not contain sufficient information

Faithfulness validation: Automated faithfulness scoring (using an LLM-as-judge or a dedicated faithfulness model) is applied to production responses and logged

Confidence thresholds: Responses below a defined faithfulness threshold are flagged for human review rather than returned directly to the end user

No-answer handling: The system is explicitly designed to return a "cannot answer from available documents" response when retrieval does not surface sufficient relevant content — rather than generating a response unsupported by the retrieved context

11.11 RAG System Evaluation

RAG systems are evaluated on the following dimensions using a curated evaluation dataset of question-answer pairs with known source documents:

Dimension	Metric	Minimum Target
Retrieval Recall@k	Fraction of queries where the correct source is in top-k	≥ 0.85 @ k=5
Answer Faithfulness	Fraction of claims in answer supported by retrieved context	≥ 0.90
Answer Relevance	Degree to which the answer addresses the query	≥ 4.0/5.0 (human rating)
Source Attribution Accuracy	Fraction of answers with correct source citation	≥ 0.95
Latency (P95)	End-to-end query response time	≤ 5 seconds

Evaluation is conducted on at least 100 query-answer pairs before production deployment. Evaluation results are stored in the project documentation folder and referenced in the deployment approval.

Section 12

Prompt Engineering Best Practices

12.1 Prompts as Versioned Artefacts

At CoreAxis, prompts are not inline strings — they are versioned engineering artefacts that are subject to review, testing, and change management. Every prompt used in a production AI feature is stored as a versioned template file in the project repository.

The naming convention for prompt files is: `<feature_name>_<prompt_role>_v<N>.txt`. For example: `document_qa_system_v3.txt`, `summary_user_v1.txt`.

Changes to production prompts follow the same code review and change management process as application code changes. No prompt change may go into production without a peer review and a documented evaluation of the change's impact.

12.2 Prompt Structure Standards

Production prompts at CoreAxis are structured using the following components, applied as appropriate to the use case:

Role / Persona: Defines the model's identity and scope. Example: "You are an expert technical support assistant for CoreAxis Technologies enterprise software." Roles must be precise — vague personas produce inconsistent behaviour.

Task Instructions: A clear, unambiguous statement of what the model must do. Instructions must be positive (specify what to do) rather than purely negative (avoid lists of what not to do).

Context: Retrieved documents, user history, or structured data provided to the model. Context is clearly delimited using XML-style tags or triple backtick blocks.

Output Format: An explicit specification of the required output format (JSON schema, numbered list, specific headings, etc.). Where structured output is required, a concrete example is provided.

Constraints: Safety, tone, and scope constraints. Example: "Respond only in the language of the user's query." Keep constraints specific and enforceable.

Examples (Few-Shot): For complex tasks, two to five carefully selected input/output examples demonstrating the desired behaviour. Examples must be representative of production inputs.

12.3 Prompt Testing

Before any prompt is used in a staging or production environment, it must pass a documented prompt test suite. The test suite includes:

Happy path tests: Standard inputs that should produce correct, well-formatted outputs

Edge case tests: Unusual but valid inputs (very long queries, queries in unexpected languages, queries with special characters)

Adversarial tests: Prompt injection attempts, jailbreak attempts, requests designed to elicit refusals or unsafe outputs

Format validation tests: Inputs where the output format can be programmatically verified

Test results are documented and stored alongside the prompt version file.

12.4 Prompt Injection Prevention

Prompt injection — the insertion of adversarial instructions into user inputs or external content that causes the model to deviate from intended behaviour — is a significant security concern for LLM applications. CoreAxis requires the following controls:

User inputs are never directly concatenated into the system prompt. User content is always placed in a clearly delimited user message or context block

Ingested document content is sanitised and clearly delimited before inclusion in prompts

System prompts include explicit instructions to ignore conflicting instructions embedded in user content or retrieved documents

Output validation is applied to detect unusual model behaviour that may indicate successful injection

12.5 Temperature and Sampling Parameters

Temperature and sampling parameters are documented for every production prompt. General guidance:

For factual, structured, or analytical tasks: temperature ≤ 0.3

For creative, conversational, or generative tasks: temperature 0.5–0.8

Top-p (nucleus sampling) should be set in conjunction with temperature; 0.9 is a reasonable default for most tasks

Max tokens must always be set explicitly and matched to the expected output length

Parameters are stored in the prompt configuration file alongside the template text, not in application configuration files where they may be changed without a corresponding prompt review.

Section 13

Embedding Model Selection Guidelines

13.1 Role of Embedding Models

Embedding models are the core of semantic search and retrieval in RAG systems. The choice of embedding model determines the quality of semantic representation, the dimension of the vector index, and the query latency of retrieval operations. Embedding model selection must be treated as a foundational architectural decision, not a default configuration.

13.2 Selection Criteria

When selecting an embedding model for a new RAG system or semantic search feature, the following criteria must be evaluated and documented:

Domain Fit: General-purpose embedding models perform well on broad tasks. Domain-specific models (legal, medical, code, multilingual) may significantly outperform on relevant corpora. Domain fit should be validated on a sample of the target corpus before committing to a model.

Benchmark Performance: Reference the MTEB (Massive Text Embedding Benchmark) leaderboard for comparative performance data. Models should be evaluated on benchmark tasks that are close in structure to the production task.

Embedding Dimension: Higher-dimensional embeddings capture more semantic nuance but increase storage and retrieval cost. Match dimension to the system's performance and cost requirements.

Context Length: The model's maximum input token length must accommodate the chunk sizes used in the RAG pipeline. Chunks that exceed the model's context length are silently truncated in most implementations — this must be handled explicitly.

Inference Latency: Evaluate both query embedding latency (per-query) and corpus embedding throughput (batch). High query latency directly impacts end-user response time.

Hosting Model: Is the model available via API (e.g. OpenAI, Cohere), as a self-hosted model from Hugging Face, or as a commercial offering? Self-hosted models avoid data privacy concerns with API providers but require infrastructure management.

Licence: Open-source models must have a licence that permits commercial use. This must be verified and documented before the model is used in any client-facing system.

13.3 Approved Embedding Models

The following categories of embedding models are approved for production use at CoreAxis. The exact model version must be documented in the system's architecture document and pinned in the dependency configuration:

Category	Example Models	Typical Use

General purpose, self-hosted	sentence-transformers/all-MiniLM-L6-v2, BAAI/bge-large-en-v1.5, intfloat/e5-large-v2	General RAG, knowledge bases
Multilingual, self-hosted	sentence-transformers/paraphrase-multilingual-mpnet-base-v2, intfloat/multilingual-e5-large	Multi-language document corpora
Code embedding	microsoft/codebert-base, Salesforce/SFR-Embedding-Code	Code search, technical documentation
Commercial API	text-embedding-3-large (OpenAI), embed-english-v3.0 (Cohere)	When self-hosting is not feasible; subject to data policy review

13.4 Embedding Model Versioning

Because the embedding space changes with each model version, the embedding model version is immutable for a given vector index. Any upgrade to the embedding model requires a planned re-ingestion event. The re-ingestion plan must be documented, approved by the Team Lead, and include a validation step confirming that retrieval quality is maintained (or improved) on the evaluation query set.

The embedding model identifier (model name and version) is stored as metadata in the vector database collection configuration and in the project's technical architecture document.

13.5 Embedding Quality Validation

Before an embedding model is used in production, its quality must be validated on a representative sample of the target corpus and query set. Validation confirms:

Nearest neighbour quality: manual inspection of top-5 results for a set of representative queries confirms semantic relevance

Retrieval recall on the evaluation query set meets the target defined in Section 11.11

No systematic failures on important edge cases (short queries, technical jargon, entity-heavy queries)

Section 14

Vector Database Standards

14.1 Overview

Vector databases are the storage and retrieval backbone of all RAG systems and semantic search features at CoreAxis. This section defines the standards for vector database selection, configuration, operation, and maintenance.

14.2 Approved Vector Databases

The following vector databases are approved for use at CoreAxis. Selection is based on the project's scale, deployment environment, and retrieval requirements:

Database	Best Suited For	Deployment
ChromaDB	Development, prototyping, small-scale production (≤ 1M vectors)	Local, Docker, self-hosted
FAISS	High-throughput in-memory similarity search; offline batch retrieval	Embedded in service, not a standalone DB
pgvector (PostgreSQL)	Systems where data already lives in PostgreSQL; lower operational overhead	PostgreSQL extension; cloud or self-hosted
Pinecone	Large-scale production (multi-million vectors), managed fully	Managed cloud service; requires Security review

No vector database other than those listed above may be used in a production system without written approval from the Department Head.

14.3 Index Configuration Standards

Vector index configuration must be documented in the system's architecture document. The following parameters must be explicitly set and justified:

Distance Metric: Cosine similarity is the default for text embeddings. Inner product (dot product) is used for normalised embeddings where it is equivalent to cosine similarity. Euclidean distance is used only when specifically justified.

HNSW Parameters (for ChromaDB, pgvector): `M` (number of connections per node) and `ef_construction` (construction-time search width) are tuned based on corpus size and query recall requirements. Default values must not be assumed adequate for production — they must be validated.

Index Size Estimation: Pre-ingestion estimates of index size, RAM requirements, and query latency must be documented before infrastructure is provisioned.

14.4 Operational Requirements

Backups: All production vector indices must be backed up at least daily. Backup procedures must be documented and tested before the system goes live.

Version Control of Index Schemas: The metadata schema and index configuration are version-controlled. Schema changes follow the same change management process as application code changes.

Monitoring: Query latency, index size growth, and query throughput are monitored and alerted. Alerts are configured for latency degradation and unexpected query failures.

Access Control: Vector database access is restricted using role-based access control. Application services access the database using service accounts with minimal required permissions. Admin access is restricted to MLOps Engineers.

14.5 FAISS-Specific Standards

FAISS is used as an embedded library within Python services, not as a standalone database. When using FAISS in production:

- The FAISS index is serialised and stored in object storage (AWS S3 or Azure Blob Storage) after each update
- The service loads the FAISS index into memory on startup — startup time and memory footprint must be accounted for in the deployment configuration
- Index updates (new document ingestion) require a service restart or an online update mechanism — the strategy must be documented
- FAISS indices larger than 10 GB must be evaluated for transition to a dedicated vector database service

14.6 Data Isolation

In multi-tenant RAG systems, client data must be isolated at the collection or namespace level. A single shared collection containing interleaved data from multiple clients is not acceptable. Isolation strategy (separate collections, separate indices, or row-level namespace filtering) must be documented and validated in the security review before production deployment.

Section 15

Model Evaluation Guidelines

15.1 Evaluation Philosophy

Model evaluation at CoreAxis is not the final step — it is an ongoing practice. We evaluate models before deployment to establish production readiness, and we evaluate them continuously in production to detect degradation. A model that passes a one-time evaluation but is never evaluated again in production is not considered responsibly deployed.

All evaluation results must be reproducible. Evaluation code is committed to the project repository and executed in a consistent, versioned environment. Evaluation metrics are logged in MLflow alongside training metrics.

15.2 Evaluation Dataset Standards

Evaluation must be conducted on a held-out test set that has had no role in any training or tuning decision. The test set must be representative of the production data distribution. If the production distribution is expected to shift over time, a temporal split (train on older data, test on more recent data) is preferred over a random split.

Minimum test set sizes by task type:

Task Type	Minimum Test Set Size
Binary classification	1,000 examples (balanced, or with documented class distribution)
Multi-class classification	200 examples per class (minimum)
Regression / ranking	500 examples
Named entity recognition	500 sentences / 100 documents
LLM generation quality	100 human-rated examples minimum
RAG end-to-end evaluation	100 question-answer pairs with known source documents

15.3 Standard Metrics by Task Type

Classification

Accuracy, Precision, Recall, F1 (macro and weighted) — reported for all classes

Confusion matrix (stored as evaluation artefact)

ROC-AUC and PR-AUC for binary and probabilistic classifiers

Calibration curve — especially for models used in risk-scoring contexts

Regression

MAE, RMSE, MAPE — reported on the test set

Residual distribution analysis (saved as a plot artefact)

R^2 — reported alongside residual metrics

Information Retrieval / RAG

Recall@k, MRR (Mean Reciprocal Rank), NDCG for retrieval

Answer faithfulness, answer relevance, contextual precision for generation

RAGAS framework is the preferred evaluation framework for RAG-specific metrics

LLM Generation

Human evaluation using a structured rubric (accuracy, coherence, safety, fluency — each on a 1–5 scale)

Automated metrics (ROUGE-L, BERTScore) as supplementary signals, not primary evaluation criteria

LLM-as-judge evaluation (secondary model evaluating primary model outputs) is acceptable when human evaluation is not feasible, subject to documented validation of the judge's agreement with human raters

15.4 Baseline Comparisons

Every evaluation report must include a comparison against at least one baseline. Acceptable baselines include: the existing production model (for model replacements), a simple rule-based system, a classical ML model, or a simpler model variant. A complex model must outperform its baseline by a meaningful margin to justify its additional cost and maintenance overhead.

15.5 Error Analysis

Error analysis — systematic inspection of the model's failure cases — is a required component of every model evaluation before production approval. Error analysis identifies: the types of errors being made, whether errors cluster in particular data

subgroups, and whether errors have disproportionate real-world consequences. Findings from error analysis must be documented in the Model Evaluation Report.

15.6 Model Evaluation Report

The Model Evaluation Report is a structured document that records all evaluation results and must be completed and signed off before a model is promoted to production. The report includes: test set description, evaluation metrics results, baseline comparison, error analysis findings, fairness assessment (see Section 16), and a recommendation (Approve, Approve with Conditions, or Reject).

Section 16

Responsible AI Guidelines

16.1 Our Commitment

CoreAxis Technologies is committed to building AI systems that are fair, transparent, explainable, and safe. Responsible AI is not a compliance checkbox — it is an engineering discipline that must be embedded throughout the AI development lifecycle. Every member of the AI & ML Department shares responsibility for the ethical quality of the systems they build.

16.2 Fairness

AI systems built at CoreAxis must not produce systematically different outcomes for individuals or groups based on protected attributes such as gender, ethnicity, age, religion, disability, or socioeconomic status, unless such differentiation is explicitly required and legally justified by the use case.

Required Actions:

- Identify all protected attributes relevant to the task domain at the start of each project

Evaluate model performance disaggregated by protected attributes on the evaluation dataset

Report disparate impact metrics (e.g. demographic parity difference, equalised odds) in the Model Evaluation Report

If statistically significant performance disparities are found, the team must investigate root causes and implement mitigations before the model is approved for production

Fairness constraints must be documented and agreed with relevant stakeholders before the model is deployed. Acceptable performance gaps across demographic groups must be specified in the project requirements — not determined post-hoc.

16.3 Transparency and Explainability

Users and affected parties have the right to understand, at an appropriate level of detail, how AI-powered systems make decisions that affect them. CoreAxis implements transparency and explainability at two levels:

System-Level Transparency: A public-facing or client-facing description of the AI system's purpose, the types of data it uses, and the nature of the decisions it supports is required for all client-deployed AI systems. This description must not be misleading about the system's capabilities or limitations.

Decision-Level Explainability: For models that support individual decisions (credit scoring, fraud detection, content moderation, hiring recommendations, etc.), individual prediction explanations must be available. Approved explainability methods include SHAP, LIME, and attention-based visualisations for neural models. The explainability method must be validated against ground truth where possible.

16.4 Human Review

AI systems at CoreAxis are designed to augment human decision-making, not unconditionally replace it. The following human review requirements apply:

For high-stakes decisions (medical, legal, financial, employment), AI outputs are advisory — a qualified human reviews the AI recommendation before action is taken

All RAG and LLM systems that return responses below a defined confidence or faithfulness threshold route those responses to a human reviewer rather than returning them directly to the end user

Human override mechanisms must be present in any AI-assisted workflow. The system must log all instances where a human overrides the AI recommendation, and these logs must be reviewed periodically to detect systematic errors

16.5 Bias Mitigation

Bias in AI systems can arise from the training data, the feature engineering process, the model architecture, or the evaluation methodology. At CoreAxis, bias mitigation is applied at multiple points:

Data-Level: Audit training data for under-representation of protected groups; apply resampling, re-weighting, or targeted data collection to address significant imbalances

Preprocessing-Level: Remove or transform features that act as proxies for protected attributes when their use cannot be justified

Model-Level: Fairness constraints may be incorporated as regularisation terms during training where technically feasible

Post-Processing-Level: Threshold calibration by demographic group may be applied to achieve equitable classification rates across groups, where this is legally permissible

16.6 Privacy and Data Protection

All AI development activities must comply with applicable data protection regulations, including but not limited to India's Digital Personal Data Protection Act (DPDPA). Specific requirements:

- Personal data used in model training must be minimised to what is necessary for the task
- Training data containing personal information must be anonymised or pseudonymised where technically feasible without degrading model performance below acceptable thresholds
- Models must be tested for memorisation of training data. Any model that demonstrably reproduces personal information from training data must not be deployed without explicit mitigation
- Data subject access and deletion rights must be considered in the system design — the team must have a documented procedure for responding to a request to delete an individual's data from both the training dataset and any derived model

16.7 AI Governance

AI governance at CoreAxis is maintained through the following structures:

- **AI Ethics Review:** Projects assessed as high-risk (as defined in the AI Risk Classification Framework) must undergo an AI Ethics Review conducted by the Department Head and a representative from Information Security before production deployment
- **Responsible AI Checklist:** Every model evaluation must include a completed Responsible AI Checklist, which covers fairness assessment, explainability provision, human review design, and privacy compliance. The checklist is stored in the project documentation folder.

Periodic Audit: Production AI systems are audited for responsible AI compliance on an annual basis or following any significant change to the model or the data it uses

Section 17

AI Security Requirements

17.1 Security as a First-Order Requirement

AI systems introduce unique security attack surfaces that differ from traditional software. Model extraction, adversarial inputs, prompt injection, data poisoning, and training data theft are real threats that must be addressed in every production AI system at CoreAxis. Security is designed in — not audited in.

17.2 Threat Modelling for AI Systems

Every AI system that will be deployed to production must undergo a formal threat modelling exercise before the deployment review. Threat modelling for AI systems identifies and documents:

Attack surface components: API endpoints, model inputs, data pipeline ingestion points, admin interfaces

Relevant threat actors: external adversaries, insider threats, automated scanners

AI-specific threats: prompt injection, adversarial inputs, model extraction via queries, training data poisoning

Mitigations in place for each identified threat

Residual risks and their acceptance status

17.3 Input Validation and Sanitisation

All inputs to AI services — whether from users, other services, or ingested documents — are validated and sanitised before processing. Requirements:

Input length is bounded and enforced before tokenisation

Input character sets are validated where applicable

File uploads for ingestion pipelines are scanned for malware before processing

Inputs are logged with sufficient detail to enable forensic reconstruction of an attack, but PII in inputs is filtered before log storage

17.4 Model Security

Model Artefact Integrity: Model artefacts in the registry are stored with checksums. Before a model is loaded for inference, its checksum is verified. Any mismatch causes the deployment to abort and an alert to be raised.

Model Access Control: Trained model weights and artefacts are stored in access-controlled storage. Access is restricted to the MLOps team and automated deployment pipelines. Individual engineers do not have direct read access to production model weights without team lead approval.

Adversarial Robustness Testing: For models that process user-provided inputs (images, text, documents), adversarial robustness testing is conducted as part of the evaluation phase. The test suite and results are stored in the project evaluation folder.

17.5 API Security

All AI services expose their endpoints over HTTPS. HTTP is not permitted in any staging or production environment.

API endpoints are authenticated using API keys or JWT tokens. Unauthenticated endpoints are not permitted in production.

Rate limiting is implemented at the API gateway level for all public-facing AI endpoints.

Output filtering is applied to all LLM API responses to detect and block outputs containing PII, credentials, or content that violates the application's content policy.

17.6 Data Pipeline Security

All data in transit between pipeline components is encrypted using TLS 1.2 or higher.

All data at rest (datasets, model artefacts, vector indices) is encrypted using AES-256.

Data access events are logged centrally in the organisation's SIEM system.

Credentials (API keys, database passwords) are never hardcoded in code or committed to Git. All credentials are managed through the organisation's secrets manager (AWS Secrets Manager or Azure Key Vault).

17.7 Third-Party Model and Library Risk

Open-source models and AI libraries introduce supply chain risk. CoreAxis applies the following controls:

Dependency versions are pinned and stored in the project repository. Unpinned dependencies are not permitted in production code.

New third-party AI libraries require a security assessment before they are added to any production project. The assessment covers licence, maintainer history, known vulnerabilities, and data handling.

Models downloaded from Hugging Face or other public repositories are verified via checksum against the official repository before they are used in production.

Unofficial model uploads or forked models require explicit sign-off from Information Security.

Section 18

AI Model Versioning

18.1 Versioning Principles

Consistent model versioning enables reproducible deployments, rollback capability, and clear audit trails. At CoreAxis, model versioning is enforced through MLflow and follows

a documented convention. Unversioned models may not be deployed to any environment.

18.2 Model Version Naming Convention

Models are assigned a semantic version following the convention:

MAJOR.MINOR.PATCH

> **MAJOR:** Incremented when the model architecture changes, the task definition changes, or the model is retrained on a fundamentally different dataset. A MAJOR version change requires a full evaluation cycle.
> **MINOR:** Incremented when the model is retrained on updated data or when a hyperparameter change produces measurably improved performance. A MINOR version change requires an evaluation on the standard test set.
> **PATCH:** Incremented for bug fixes in preprocessing, parameter corrections, or configuration changes that do not affect model weights. A PATCH version change requires a smoke test and peer review.

Model names follow the convention: `<project_id>_<model_name>`. For example: `contract_review_bert_classifier`, `support_qa_rag_pipeline`.

18.3 MLflow Model Registry

All models produced for production use are registered in the MLflow Model Registry. The registry is the authoritative source of record for model versions and their deployment status. Direct deployment from a local training environment to production — bypassing the registry — is not permitted.

The model registry uses the following lifecycle stages:

Stage	Definition	Permissions
None	Model is logged but not yet evaluated	Any engineer
Staging	Model has passed evaluation; awaiting staging validation	ML Engineer with Team Lead approval
Production	Model is currently serving production traffic	MLOps Engineer with Team Lead approval
Archived	Model has been retired from production	MLOps Engineer

18.4 What Is Versioned

A model version in the registry includes the following artefacts:

- Serialised model weights / pipeline (in the MLflow artefact store)
- Feature preprocessor(s) with fitted parameters
- Input schema file
- Evaluation metrics (logged as MLflow metrics)
- Model Card (linked document)
- Git commit hash of the training code
- Dataset version identifier

18.5 Rollback Policy

The previous Production stage model version is retained in the Archived state for a minimum of 90 days following a production promotion. During this window, the previous version can be promoted back to Production in the event of a rollback. The rollback procedure is documented in the system's Deployment Runbook (see Section 20).

Rollback is initiated by the MLOps Engineer at the direction of the Team Lead. Any production rollback event must be followed by a post-incident review to identify and address the root cause of the issue that necessitated the rollback.

18.6 Data and Code Versioning

Model versioning is not sufficient in isolation. Complete reproducibility requires co-versioning of:

Data: Dataset version identifiers are linked to every model version. See Section 6.4.

Code: The Git commit hash of the training code is logged in every MLflow run and registered model version.

Environment: The Docker image tag used for training is stored alongside the model artefact.

Section 19

Experiment Tracking

19.1 Why Experiment Tracking is Mandatory

Machine learning development involves continuous experimentation — varying data, features, architectures, and hyperparameters to find the best-performing solution. Without systematic tracking, experiments are not reproducible, comparisons between approaches are unreliable, and knowledge accumulated during development is lost. CoreAxis mandates structured experiment tracking for all ML development using MLflow.

19.2 MLflow Setup and Organisation

Each project has a dedicated MLflow experiment, named using the convention: `<project_id>_<experiment_name>`. Engineers working on a project log all training

runs to that project's experiment. Cross-project experiments are not permitted — each experiment folder belongs to exactly one project.

MLflow is hosted on the organisation's shared ML platform. Connection settings and access credentials are provisioned by the MLOps team upon project setup. Engineers must not run local-only MLflow instances for production experiments — all logged runs must be visible in the shared platform.

19.3 Required Logged Artefacts Per Run

Every training run that is intended to produce a model candidate must log the following in MLflow:

Parameters

- All hyperparameters used in the run (learning rate, batch size, number of epochs, regularisation values, architecture choices, etc.)
- Dataset version identifier
- Random seed(s)
- Model architecture name and variant
- Git commit hash of training code

Metrics

- Training and validation loss at each epoch (or at documented intervals for large models)
- All evaluation metrics from the test set at the end of training
- Training duration (wall-clock time)
- Peak GPU/CPU memory usage (where instrumentation is available)

Artefacts

- Serialised model (logged via `mlflow.<framework>.log_model`)
- Training configuration file (YAML or JSON)
- Feature importance plot or SHAP summary plot (where applicable)

Confusion matrix or other task-specific evaluation visualisations

Learning curves (training and validation loss over epochs)

19.4 Run Naming and Tagging

Runs are named descriptively to support later comparison: `<model_type>_<key_variation>_<YYYYMMDD>`. For example: `bert_base_lr2e5_batch32_20250601`. Tags are applied to label run purpose (baseline, hyperparameter_search, ablation, final_candidate) and to link to the relevant GitHub issue or project task ID.

19.5 Exploratory vs Production Runs

Exploratory runs (quick experiments during development) are tagged with `run_type: exploratory`. They may have incomplete logging. Production candidate runs are tagged `run_type: production_candidate` and must meet all logging requirements before they are considered for model registration. Only production candidate runs may be registered in the MLflow Model Registry.

19.6 Experiment Reviews

At the end of the development phase, before a model is submitted for evaluation, the team conducts an experiment review. In this review, the team examines the experiment's run history, identifies the best-performing configurations, and documents the reasoning for selecting the production candidate. The experiment review findings are summarised in the Model Evaluation Report.

Section 20

Model Deployment Process

20.1 Deployment Principles

Deployment of AI models is a controlled process with defined gates, documented procedures, and rehearsed rollback plans. No model may be deployed to production

without completing the required pre-deployment steps. Deployment is the responsibility of the MLOps team, executed in coordination with the owning AI/ML team.

20.2 Pre-Deployment Requirements

The following must be completed before a production deployment is initiated:

- Model registered in MLflow Model Registry at the Staging stage
- Signed Model Evaluation Report on file
- Completed Responsible AI Checklist on file
- Security threat model reviewed and approved by Information Security (for new systems or significant changes)
- Deployment Runbook written and reviewed
- Rollback plan documented
- Monitoring and alerting configuration ready (to be activated at deployment time)
- Staging validation completed and signed off by Team Lead

20.3 Containerisation Standards

All production AI services are deployed as Docker containers. Container images must meet the following standards:

- Built from an approved base image (organisation-maintained images based on official Python or CUDA base images)
- No unnecessary packages installed — minimal attack surface
- No secrets, credentials, or hardcoded environment values inside the image
- Image layers are ordered to maximise build cache efficiency
- Images are scanned for known vulnerabilities (CVEs) using the CI pipeline's container scanning step before they are pushed to the registry
- All images are tagged with the model version and Git commit hash: `<model_name>:<model_version>-<git_short_sha>`

20.4 Deployment Environments

CoreAxis maintains three deployment environments for AI services:

Environment	Purpose	Access
Development	Active development and unit testing	Individual engineers
Staging	Integration testing, load testing, stakeholder review	Team; limited external access
Production	Live client traffic	Controlled; MLOps team manages deployments

Promotion from Staging to Production requires a formal sign-off from the Team Lead and confirmation of successful staging validation. Hotfixes may be deployed to production following an expedited review process with approval from both the Team Lead and the Department Head.

20.5 Kubernetes Deployment Configuration

Production AI services are orchestrated using Kubernetes. Deployment configurations (manifests or Helm charts) are committed to the project repository under a `/deploy` directory. Required configuration elements include:

Resource requests and limits (CPU, memory, GPU where applicable) — sizing is based on load test results from the staging environment

Liveness and readiness probes — readiness probe confirms the model is loaded and serving before the deployment receives traffic

Horizontal pod autoscaling (HPA) configuration with documented scaling triggers

Secret references via Kubernetes Secrets (mounted from cloud secrets manager, never stored in plaintext)

Node affinity rules for GPU workloads

20.6 API Service Standards

AI model capabilities are exposed via FastAPI services. Service design requirements:

- A `/health` or `/healthz` endpoint returning model load status and version
- A `/predict` (or semantically equivalent) endpoint with documented request/response schemas using Pydantic models
- Input validation returning 422 for malformed requests with field-level error details
- Structured logging of each request: timestamp, model version, input hash (not raw input), latency, output class/confidence
- OpenAPI documentation auto-generated by FastAPI and accessible at `/docs` in non-production environments

20.7 Deployment Runbook

Every production AI deployment must have a documented Deployment Runbook. The runbook covers: pre-deployment checklist, deployment command sequence, post-deployment validation steps, smoke test procedure, rollback trigger conditions, rollback command sequence, and escalation contacts. The runbook is reviewed and rehearsed before the first production deployment of a new model.

20.8 Gradual Rollout

For models that replace an existing production model, a gradual rollout strategy is used where operationally feasible. Traffic is routed incrementally to the new model version (e.g. 5% → 25% → 100%) with monitoring checks between each increment. If monitoring indicates degradation at any increment, the rollout is paused and evaluated before proceeding.

Section 21

Model Monitoring and Maintenance

21.1 Monitoring Philosophy

A model deployed to production without monitoring is not responsibly deployed. All production AI models at CoreAxis are monitored continuously for operational health, prediction quality, and data distribution changes. Monitoring is configured before the model goes live and is a prerequisite for production deployment approval.

21.2 Operational Monitoring

Operational monitoring covers the technical health of the AI service. The following metrics are monitored and alerted on for all production AI services:

Metric	Alert Threshold
API latency (P95)	> 2× baseline for > 5 minutes
Error rate	> 1% of requests
Service availability	< 99.5% over any 1-hour window
Memory usage	> 85% of allocated memory limit
Throughput (requests/minute)	> 3× or < 0.1× baseline
Model loading time	> 120 seconds on pod start

Alerts are routed to the on-call MLOps Engineer and the project Team Lead via PagerDuty and Slack.

21.3 Data Drift Detection

Data drift — changes in the statistical distribution of input data over time — is one of the most common causes of model performance degradation in production. CoreAxis monitors for data drift using the following approach:

A reference distribution is established from the training/validation dataset at deployment time

The distribution of production inputs is measured daily and compared to the reference distribution using statistical tests (Kolmogorov-Smirnov test for continuous features, chi-squared test for categorical features)

Drift alerts are triggered when the p-value drops below 0.05 for any monitored feature, or when the overall drift score exceeds the documented threshold

Drift events are logged and reviewed by the owning ML Engineer

21.4 Prediction Drift Detection

Prediction drift — changes in the distribution of model outputs — is monitored separately from input drift. Significant shifts in the distribution of predicted classes or scores may indicate model degradation even before labelled ground truth is available. Prediction distribution is monitored daily; alerts are triggered on significant distributional shifts.

21.5 Ground Truth Monitoring

Where ground truth labels become available with a delay (e.g. fraud labels after transaction review, churn labels after the customer observation period), actual model performance is evaluated against ground truth on a regular schedule. The monitoring schedule and the process for incorporating delayed labels are documented in the Deployment Runbook.

21.6 Retraining Triggers

Model retraining is initiated when one or more of the following conditions is met:

Performance on ground truth labels degrades below the documented minimum acceptable threshold

A significant drift event is sustained for more than 7 days

The scheduled periodic review (every 6 months) identifies a meaningful performance gap relative to current data

A significant change in the upstream data pipeline or feature definitions occurs

A new version of a dependent model (e.g. embedding model, base LLM) requires retraining for compatibility

Retraining follows the same development lifecycle as the original model — it is not an expedited process. The retrained model must be evaluated against the same metrics and thresholds before it replaces the production model.

21.7 Monitoring for LLM and RAG Systems

In addition to standard operational monitoring, LLM and RAG systems require the following specific monitoring:

Faithfulness Scoring: A sample of production responses is scored for faithfulness against the retrieved context on a daily basis. Sustained faithfulness decline triggers an investigation and potential pipeline change.

User Feedback Integration: User feedback signals (thumbs up/down, corrections, flagged responses) are collected and reviewed weekly. Patterns in negative feedback are triaged and addressed.

Prompt Anomaly Detection: Unusual input patterns that may indicate prompt injection attempts are flagged for security review.

Token Usage Tracking: Token consumption per request is tracked and compared to expected ranges. Significant deviations may indicate prompt manipulation or input anomalies.

Section 22

AI Incident Response

22.1 Definition of an AI Incident

An AI incident at CoreAxis is any event in which a production AI system produces outputs or behaviours that cause or risk causing harm to clients, end users, the organisation, or third parties. This includes:

- Model outputs that are factually incorrect and have influenced a consequential decision
- Discriminatory or biased outputs that have been surfaced to end users
- Safety filter failures that allowed harmful, offensive, or restricted content to be generated
- Prompt injection attacks that caused the model to deviate from intended behaviour
- Data leakage: exposure of training data, client data, or internal information via model outputs
- Service unavailability or severe latency degradation affecting client-facing AI features
- Incorrect model predictions that have led to material adverse outcomes for clients

22.2 Incident Severity Classification

Severity	Description	Response Time
P1 — Critical	Client or user harm has occurred or is imminent; data breach suspected; system is causing active discriminatory outcomes	Immediate response; escalation within 15 minutes
P2 — High	Significant performance degradation affecting all users; systematic incorrect outputs at scale; safety filter failure	Response within 1 hour
P3 — Medium	Partial performance degradation; isolated incorrect outputs; monitoring alert requiring investigation	Response within 4 hours (business hours)

P4 — Low	Non-critical anomalies; minor output quality issues; low-impact technical errors	Response within next business day

22.3 Incident Response Procedure

Step 1: Detection and Triage (0–30 minutes)

The incident is detected via monitoring alert, user report, or internal identification. The on-call engineer is notified. An initial triage is performed to classify the severity and identify the affected system. A dedicated incident Slack channel is opened and the Team Lead is notified.

Step 2: Containment (30 minutes – 2 hours)

For P1 and P2 incidents, the first priority is containment — stopping the immediate harm. Containment actions may include: rolling back to the previous model version, disabling the affected AI feature, routing all traffic to a fallback system, or temporarily suspending user access to the affected capability. Containment action and rationale are logged in the incident channel.

Step 3: Investigation

The owning team investigates the root cause using: production logs, monitoring data, MLflow experiment history, and input/output samples from the incident window. The investigation must identify the failure mode, the extent of impact (number of users or decisions affected), and whether the issue represents a data problem, model problem, infrastructure problem, or prompt/configuration problem.

Step 4: Remediation

A remediation plan is developed and approved by the Team Lead before implementation. Remediation may involve: a model hotfix and redeployment, a configuration change, a data pipeline correction, or a prompt update. All remediation changes follow the standard change management process.

Step 5: Post-Incident Review

Within five business days of resolution, a Post-Incident Review (PIR) is conducted by the owning team. The PIR produces a written report covering: timeline of events, root cause analysis, impact assessment, remediation steps taken, and preventive measures to reduce the likelihood of recurrence. PIR reports are stored in the project documentation folder and reviewed by the Department Head.

22.4 Client Notification

Where an AI incident has affected client-facing services or client data, the Department Head and the Account Management team must be notified immediately. Client communication regarding AI incidents follows the organisation's client communication protocol. The AI team must provide factual, accurate information for client communications but must not communicate directly with clients without Account Management coordination.

22.5 Regulatory Reporting

AI incidents that constitute a personal data breach must be reported to the appropriate regulatory authority in accordance with applicable data protection law (DPDPA in India). The Information Security team leads regulatory reporting; the AI team provides technical detail as required. Regulatory notification timelines are defined in the Information Security Incident Response Policy.

Section 23

Documentation Requirements

23.1 Documentation as Engineering Practice

Undocumented AI systems are a liability. They cannot be audited, maintained, or safely extended. At CoreAxis, documentation is a first-class engineering deliverable — it is not optional, and it is not produced retroactively after the system is built. Documentation

evolves alongside the system and is maintained as a living record of what the system does, why it was built the way it was, and how to operate it safely.

23.2 Required Documents by Project Phase

Phase	Required Document	Owner
Problem Definition	Project Brief	Team Lead
Data Acquisition	Data Card	ML Engineer / Data Scientist
Data Preparation	Data Preparation Report	ML Engineer / Data Scientist
Annotation	Annotation Plan, Annotation Guidelines Document, IAA Report	ML Engineer
Feature Engineering	Feature Specifications	ML Engineer
Model Development	Experiment Log (MLflow export)	ML Engineer
Evaluation	Model Evaluation Report, Responsible AI Checklist	ML Engineer + Team Lead
Deployment	Model Card, Deployment Runbook, Architecture Document	MLOps Engineer + AI Engineer
Production	Monitoring Dashboard, Incident Log (ongoing)	MLOps Engineer

23.3 Model Card Standard

A Model Card is a structured document that must be produced for every model registered in the MLflow Model Registry for production use. The Model Card follows the industry-standard format and includes:

Model Overview: Name, version, task, intended use cases, and out-of-scope use cases

Model Architecture: Architecture type, framework, parameter count (where available)

Training Data: Dataset name and version, data sources, known limitations

Performance Metrics: Evaluation metrics on the test set, disaggregated by relevant subgroups

Ethical Considerations: Known biases, fairness assessment findings, recommended human oversight level

Caveats and Limitations: Known failure modes, out-of-distribution performance expectations

Version History: Summary of changes from previous versions

Point of Contact: Owning team and contact for questions

Model Card templates are in the documentation repository at `/templates/ai/model-card.md`.

23.4 Architecture Documentation

Every production AI system must have an Architecture Document that describes the end-to-end system design, including: data flow diagram, component descriptions, external dependencies, infrastructure configuration, security controls, and integration points with other services. Architecture Documents are stored in the project documentation folder and reviewed annually.

23.5 API Documentation

All AI services expose auto-generated FastAPI documentation at `/docs` in non-production environments. For client-facing services, a maintained API reference guide is produced separately and must not disclose internal implementation details. API documentation is updated whenever the service interface changes.

23.6 Documentation Storage and Access

All project documentation is stored in the organisation's internal documentation platform under the `/ai-projects/` namespace. Documentation is organised by project ID. Access is granted to all AI department employees by default, with restricted sections for sensitive client documentation accessible only to the relevant project team.

Section 24

Collaboration with Other Departments

24.1 Cross-Department Collaboration Principles

AI systems at CoreAxis do not exist in isolation — they are integrated into products built by Software Engineering, operated on infrastructure managed by IT, protected by Information Security, and funded and governed by Finance. Effective cross-department collaboration is essential for delivering high-quality AI products on time and within policy.

The AI & ML Department approaches cross-department collaboration with the principle of clear interfaces: we document our dependencies, communicate changes proactively, and maintain defined escalation paths for blockers and conflicts.

24.2 AI & ML and Software Engineering

The Software Engineering department builds the product-level software that integrates AI capabilities developed by the AI team. The interface between these departments is the AI service API.

AI team responsibilities:

> Publish and maintain accurate OpenAPI specifications for all AI services before Software Engineering begins integration work

Notify Software Engineering at least two weeks in advance of any breaking API changes
Provide Software Engineering with a staging environment for integration testing
Support Software Engineering in diagnosing integration issues by providing relevant logs and debugging assistance

Software Engineering responsibilities:

Communicate product requirements that have AI capability implications early in the planning cycle, not after implementation has begun
Report AI service quality or behaviour issues through the standard incident channel, not via direct individual requests

24.3 AI & ML and Information Technology

The IT Department manages the organisation's core infrastructure, network, and access control systems. The AI team works with IT for:

Provisioning of development workstations and GPU resources
Network access configuration for cloud resources and external APIs
Identity and access management — user provisioning and permission requests follow the standard IT request process
VPN and secure remote access configuration

AI infrastructure requests (new cloud resources, increased resource quotas, new service accounts) must be submitted via the IT request system with a documented business justification and security classification at least five business days before the resource is required.

24.4 AI & ML and Information Security

The Information Security team is a key partner for the AI department. Their involvement is mandatory at defined points in the AI development lifecycle:

Security review for new AI systems or significant system changes before staging deployment

Review and approval of data processing agreements for new client data sources

Threat modelling participation for high-risk AI systems

Incident response lead for any security event involving AI systems

Engage Information Security early — do not approach them for a sign-off the day before a planned deployment. Security reviews that involve new data flows, external APIs, or novel system architectures require a minimum of ten business days.

24.5 AI & ML and Finance

Finance manages budget allocation and cost governance for AI projects. The AI team's interactions with Finance include:

Project Budgeting: Annual budget requests for AI projects are submitted to Finance through the standard budget planning process. Project Briefs that include cost estimates are submitted by the Department Head.

Cloud Cost Management: Monthly cloud spend reports are reviewed with Finance. Cost overruns exceeding 20% of monthly budget require a Finance notification and a remediation plan.

Third-Party API Costs: Any new third-party AI API subscription (OpenAI, Cohere, Azure AI, etc.) must be approved by Finance before the subscription is activated.

24.6 AI & ML and Human Resources

HR manages talent acquisition, employee development, and performance management for the AI team. Key HR collaboration points include:

Hiring briefs for new AI team roles are produced by the Department Head in partnership with HR

Onboarding plans for new AI team members are coordinated between HR and the Department Head

Learning and development budget requests for conferences, courses, and certifications are submitted through the HR L&D process

Section 25

Employee Responsibilities

25.1 Professional Conduct

All employees of the AI & Machine Learning Department are expected to conduct themselves in accordance with CoreAxis's Code of Conduct and Employee Handbook. In the context of AI work, this specifically includes:

Maintaining honesty about model performance — never misrepresenting evaluation results, cherry-picking metrics, or presenting partial results as comprehensive

Raising concerns about the ethical implications of assigned work through proper channels without fear of retaliation

Treating client data and internal proprietary data with the confidentiality and care they require

Not using company AI infrastructure, compute resources, or client data for personal projects or external work

25.2 Policy Compliance

All employees are personally responsible for knowing and complying with the policies in this handbook. Ignorance of a policy is not an acceptable defence for a policy violation. If any policy in this handbook is unclear, the employee must seek clarification from their Team Lead or the Department Head before proceeding.

Violations of the policies in this handbook — particularly those relating to data security, client data handling, and Responsible AI — are treated as serious disciplinary matters

and may result in disciplinary action up to and including termination, depending on the nature and impact of the violation.

25.3 Code and Data Quality

Every employee is personally responsible for the quality of the code, data, and models they produce. Peer review is a team process, but it does not transfer responsibility from the original author. Writing untested code, producing undocumented datasets, or submitting models for production without completing the required evaluation is a professional failure, not merely a process gap.

25.4 Communication and Transparency

Employees are expected to communicate proactively about blockers, risks, and delays. Surprises — especially negative ones — are minimised by early and honest communication. If a project is at risk of missing a deadline, the engineer or scientist raises this with the Team Lead at the earliest opportunity, not at the deadline itself.

Technical decisions that have significant architectural, cost, or ethical implications must be escalated to the Team Lead before they are implemented. Individual engineers do not make unilateral decisions on model architecture changes, vendor changes, or data processing changes that affect other teams or clients.

25.5 Continuous Learning Responsibility

The field of AI evolves rapidly. Employees are expected to invest in their own continuous learning (see Section 26) and to bring relevant new knowledge back to the team. Reading and sharing relevant research, attending internal knowledge-sharing sessions, and staying current with significant developments in the tools and frameworks the team uses are professional expectations, not optional activities.

25.6 IP and Confidentiality

All work product produced during employment at CoreAxis — including code, models, datasets, documentation, prompts, and research — is the intellectual property of CoreAxis Technologies. Employees must not share, publish, or open-source any work product without written approval from the Department Head and, where applicable, the client. This includes sharing architecture details, benchmark results, or methodology descriptions with external parties (including in conference papers or blog posts).

Section 26

Continuous Learning and Research

26.1 Learning Commitment

CoreAxis Technologies is committed to supporting the professional development of all AI department employees. The field moves quickly, and our ability to deliver cutting-edge AI solutions depends on our team remaining at or near the technical frontier. Learning is a shared investment between the organisation and the individual.

26.2 Individual Learning Budget

Each AI department employee has an annual learning and development budget. This budget covers: online courses and certifications (Coursera, fast.ai, DeepLearning.AI, etc.), conference attendance (domestic and, with approval, international), technical book purchases, and professional society memberships. The budget and approval process are managed through HR. Engineers are encouraged to use their full allocation each year.

26.3 Internal Knowledge Sharing

The AI team conducts the following regular internal knowledge-sharing activities:

Weekly Paper Reading Group: A 45-minute session every Thursday where one team member presents a recent research paper relevant to the department's

work. Presentations are informal and focused on practical implications. The schedule and reading list are maintained in the team's shared channel.

Monthly Tech Talks: A one-hour internal talk by a team member on a topic of their choosing — a new technique, a tool evaluation, a post-mortem, or a project retrospective. These are recorded and stored in the knowledge base.

Post-Project Retrospectives: After each significant project, a retrospective session is held to capture what worked, what did not, and what the team would do differently. Key learnings are documented in the project folder and shared with the team.

26.4 Research Engagement

AI Research Engineers are expected to maintain active engagement with the research community. For other roles, engagement with research is encouraged but calibrated to role focus. All employees are encouraged to:

- Follow key research venues relevant to their domain (NeurIPS, ICML, ICLR, ACL, EMNLP, CVPR as appropriate)
- Submit summaries of relevant papers to the team knowledge base via the `#research-notes` channel
- Identify when a research finding has practical applicability to current projects and raise it with the Team Lead

Publishing research externally (conference papers, preprints) is encouraged for Research Engineers and supported for other roles on a case-by-case basis. Any external publication must be reviewed and approved by the Department Head before submission. Publications must not disclose client information, proprietary datasets, or internal system details without explicit approval.

26.5 Skill Development Pathways

The following skill development pathways are recommended by role:

Role	Recommended Development Areas
AI Engineer	Advanced prompt engineering, LLM application architecture, production RAG system design, API design, system design
ML Engineer	Advanced model architectures, statistical learning theory, MLOps practices, model interpretability, experiment design
Data Scientist	Advanced statistical analysis, causal inference, data visualisation, business communication of technical findings
MLOps Engineer	Kubernetes advanced patterns, distributed training infrastructure, cost optimisation, ML platform design, SRE practices
Research Engineer	Foundational ML theory, reading and implementing research papers, experimental design, scientific writing

26.6 Mentoring

Senior engineers and scientists in the AI department are expected to provide mentoring to junior team members. Mentoring relationships are informal and team-managed — engineers should proactively offer support to newer colleagues, share context, and make time for code reviews that are educational rather than merely corrective.

Section 27

Appendix

A. Approved Technology Stack Reference

Category	Approved Technologies
Programming Language	Python 3.10+
API Framework	FastAPI

Containerisation	Docker, Docker Compose
Orchestration	Kubernetes (AWS EKS, Azure AKS)
Experiment Tracking	MLflow (organisation-hosted instance)
Version Control	Git, GitHub
LLM Orchestration	LangChain, LlamaIndex
Vector Databases	ChromaDB, FAISS, pgvector, Pinecone (approved)
Relational Database	PostgreSQL
Caching	Redis
Cloud Platforms	AWS, Microsoft Azure
Local LLM Runtime	Ollama
Model Hub	Hugging Face Hub
CI/CD	GitHub Actions
Monitoring	Prometheus, Grafana

B. Key Internal Contacts

Role	Contact Point
AI & ML Department Head	Internal directory / `#dept-ai-leadership`
MLOps On-Call	PagerDuty schedule (AI-MLOps rotation)
Information Security	Internal ticket system / `#infosec-requests`
IT Helpdesk	IT ticket portal

HR — AI Team Contact	Internal directory
Legal / Compliance	Internal directory (data protection queries)

C. Document Templates Reference

Template	Location in Documentation Repository
Project Brief	/templates/ai/project-brief.md
Data Card	/templates/ai/data-card.md
Data Preparation Report	/templates/ai/data-prep-report.md
Annotation Plan	/templates/ai/annotation-plan.md
Annotation Guidelines Document	/templates/ai/annotation-guidelines.md
Model Evaluation Report	/templates/ai/model-eval-report.md
Responsible AI Checklist	/templates/ai/responsible-ai-checklist.md
Model Card	/templates/ai/model-card.md
Deployment Runbook	/templates/ai/deployment-runbook.md
Post-Incident Review Report	/templates/ai/pir-report.md
RAG Architecture Document	/templates/ai/rag-architecture.md

D. AI Risk Classification Framework

AI systems are classified by risk level based on the nature of decisions they support and the populations they affect. Risk classification is determined during the Problem Definition phase and reviewed at each deployment approval.

Risk Level	Characteristics	Additional Requirements
Low	Internal tools; no client-facing outputs; easily reversible decisions; no PII processed	Standard lifecycle requirements
Medium	Client-facing; influences but does not determine consequential decisions; PII processed with consent	+ Responsible AI Checklist; + formal fairness assessment
High	Directly determines or heavily influences significant decisions (financial, employment, access to services); processes sensitive personal data	+ AI Ethics Review; + Information Security sign-off; + human review requirement; + annual audit
Critical	Decisions that are irreversible or have safety implications (medical, infrastructure, legal)	+ Legal review; + external audit; + regulatory notification where required

E. Glossary

Chunking: The process of dividing documents into smaller segments for embedding and vector database storage in RAG systems.

Data Card: A structured document describing a dataset's properties, provenance, and usage constraints.

Data Drift: A change in the statistical distribution of input features over time compared to the training distribution.

Embedding: A dense numerical vector representation of text, images, or other data used for semantic similarity computation.

Faithfulness: The degree to which a RAG system's generated answer is supported by the retrieved documents.

Fine-Tuning: The process of further training a pre-trained model on a specific dataset to adapt it to a particular task or domain.

Hallucination: The generation of factually incorrect or fabricated information by an LLM.

Hybrid Search: A retrieval strategy that combines dense (semantic) and sparse (keyword) search methods.

IAA (Inter-Annotator Agreement): A statistical measure of the consistency of labels assigned by multiple annotators to the same data.

LLM (Large Language Model): A machine learning model trained on large text corpora that can generate, summarise, classify, and reason about text.

Model Card: A standardised document describing a model's intended use, performance characteristics, limitations, and ethical considerations.

MLflow: An open-source platform for managing the ML lifecycle including experiment tracking, model versioning, and deployment.

MLOps: The practice of applying DevOps principles to ML systems — covering CI/CD for ML, monitoring, and operational management of models in production.

Model Drift / Concept Drift: Degradation in model performance resulting from changes in the relationship between input features and the target variable over time.

PEFT (Parameter-Efficient Fine-Tuning): Fine-tuning methods such as LoRA that update only a small subset of model parameters, significantly reducing compute and storage requirements.

Prompt Injection: An attack where adversarial content in user input or retrieved documents causes an LLM to deviate from its intended behaviour.

RAG (Retrieval-Augmented Generation): An AI architecture that augments LLM generation with information retrieved from an external knowledge base.

Re-Ranking: The process of re-scoring retrieved documents using a more powerful model to improve retrieval precision before generation.

Vector Database: A specialised database for storing and querying high-dimensional embedding vectors by similarity.

F. Revision History

Version	Date	Summary of Changes
1.0	June 2025	Initial release of the AI & ML Department Handbook
1.1	September 2025	Updated RAG evaluation metrics (Section 11.11); added Hybrid Search standards (Section 11.7)
1.2	January 2026	Added AI Risk Classification Framework (Appendix D); updated embedding model approval list (Section 13.3)
1.3	June 2026	Expanded LLM security requirements (Section 17); updated vector database standards to include pgvector (Section 14.2); revised monitoring thresholds (Section 21.2)